Exam 2 / Assessment Lab 2

Salman Rashid

2026-05-30

Introduction

This document provides solutions to the 6 tasks in the exam. The analysis uses dplyr for data manipulation, lubridate for dates, and ggplot2 for visualization. All steps are explained in simple terms.

```
# Load all required packages
library(dplyr)
library(lubridate)
library(ggplot2)
library(tidyr)
library(janitor)  # for clean_names() to handle messy column names

# Read the CSV file and immediately clean column names
df_orig <- read.csv("Exam 2 Input File.csv", check.names = FALSE) %>%
  clean_names()   # converts empty first column to "x", and date column to "yyyy_mm_dd_doy"

# Check the first few rows to understand the data structure
head(df_orig)
```

Step 1: Load packages and read the data

```
##   x  yyyy_mm_dd_doy    yield irrigation_demand county_name       crop
## 1 1 1980-08-27(240) 15147.19          421.8288    Okanogan Corn_grain
## 2 2 1981-08-21(233) 15926.39          427.0740    Okanogan Corn_grain
## 3 3 1982-08-16(228) 13489.79          363.3833    Okanogan Corn_grain
## 4 4 1983-08-19(231) 15263.74          382.2921    Okanogan Corn_grain
## 5 5 1984-08-22(235) 13779.91          391.8027    Okanogan Corn_grain
## 6 6 1985-08-04(216) 13920.45          445.9654    Okanogan Corn_grain
##               lat_lon
## 1 48.15625N119.71875W
## 2 48.15625N119.71875W
## 3 48.15625N119.71875W
## 4 48.15625N119.71875W
## 5 48.15625N119.71875W
## 6 48.15625N119.71875W
```

The column yyyy_mm_dd_doy contains dates like “1980-08-27(240)”. We extract the year using lubridate after removing the parenthetical part.

```r
# Add year column cleanly
df_orig <- df_orig %>%
  mutate(date_clean = gsub("\\(.*\\)", "", yyyy_mm_dd_doy),
         date = ymd(date_clean),
         year = year(date)) %>%
  select(-date_clean, -date)

# Verify the new year column
head(df_orig)
```

```
##   x  yyyy_mm_dd_doy    yield irrigation_demand county_name       crop
## 1 1 1980-08-27(240) 15147.19          421.8288    Okanogan Corn_grain
## 2 2 1981-08-21(233) 15926.39          427.0740    Okanogan Corn_grain
## 3 3 1982-08-16(228) 13489.79          363.3833    Okanogan Corn_grain
## 4 4 1983-08-19(231) 15263.74          382.2921    Okanogan Corn_grain
## 5 5 1984-08-22(235) 13779.91          391.8027    Okanogan Corn_grain
## 6 6 1985-08-04(216) 13920.45          445.9654    Okanogan Corn_grain
##               lat_lon year
## 1 48.15625N119.71875W 1980
## 2 48.15625N119.71875W 1981
## 3 48.15625N119.71875W 1982
## 4 48.15625N119.71875W 1983
## 5 48.15625N119.71875W 1984
## 6 48.15625N119.71875W 1985
```

Step 2: Remove rows with irrigation demand > 90th percentile (by county and crop) We compute the 90th percentile of irrigation_demand for each combination of county_name and crop. Then we keep rows where demand is not above that threshold.

```r
# Group by county and crop, then calculate the 90th percentile demand within each group
df_filtered <- df_orig %>%
  group_by(county_name, crop) %>%
  mutate(demand_90 = quantile(irrigation_demand, 0.9, na.rm = TRUE)) %>%
  ungroup() %>%
  # Keep rows where irrigation_demand is not above the 90th percentile
  filter(irrigation_demand <= demand_90) %>%
  # Remove the helper column demand_90
  select(-demand_90)

# Print how many rows were removed
n_removed <- nrow(df_orig) - nrow(df_filtered)
cat("Rows removed (outliers):", n_removed, "\n")
```

```
## Rows removed (outliers): 44
```

```r
cat("Original rows:", nrow(df_orig), "\n")
```

```
## Original rows: 474
```

```r
cat("Filtered rows:", nrow(df_filtered), "\n")
```

```
## Filtered rows: 430
```

Step 3: Create separate CSV files for irrigation demand and yield We retain year, county_name, crop, lat_lon, and the respective variable. We save them as CSV files.

```r
# Create a dataset containing only irrigation demand
demand_data <- df_filtered %>%
  select(year, county_name, crop, lat_lon, irrigation_demand)

# Create a dataset containing only yield
yield_data <- df_filtered %>%
  select(year, county_name, crop, lat_lon, yield)

# Write both to CSV files in the working directory
write.csv(demand_data, "irrigation_demand.csv", row.names = FALSE)
write.csv(yield_data, "yield.csv", row.names = FALSE)

# Confirm success
cat("Files created: irrigation_demand.csv and yield.csv\n")
```

```
## Files created: irrigation_demand.csv and yield.csv
```

Step 4: Read the two CSV files and perform joins We read the two files and then perform full, left, and right joins on the key columns: year, county_name, crop, lat_lon. Then we compare the number of rows.

```r
# Read the saved CSV files back into R
demand <- read.csv("irrigation_demand.csv")
yield <- read.csv("yield.csv")

# Full join: keeps all rows from both tables
full_join_df <- full_join(demand, yield, by = c("year", "county_name", "crop", "lat_lon"))

# Left join: keeps all rows from the left table (demand) and matching rows from the right
left_join_df <- left_join(demand, yield, by = c("year", "county_name", "crop", "lat_lon"))

# Right join: keeps all rows from the right table (yield) and matching rows from the left
right_join_df <- right_join(demand, yield, by = c("year", "county_name", "crop", "lat_lon"))

# Compare row counts
n_full <- nrow(full_join_df)
n_left <- nrow(left_join_df)
n_right <- nrow(right_join_df)

cat("Full join rows:", n_full, "\n")
```

```
## Full join rows: 430
```

```r
cat("Left join rows:", n_left, "\n")
```

```
## Left join rows: 430
```

```r
cat("Right join rows:", n_right, "\n")
```

```
## Right join rows: 430
```

Why are they the same? Both demand and yield were created from the same filtered dataset (df_filtered). Every row in demand has a matching row in yield because each original observation contained both variables. Therefore, all joins preserve exactly the same set of rows. No rows are lost or added because there are no unmatched keys.

Step 5: Check completeness relative to the original dataset We compare the full join result (which contains only the filtered data) with the original dataset (including outliers). The missing rows are exactly the ones we removed in Step 2.

```r
# Create a unique key for each row by combining year, county, crop, and lat_lon
df_orig_key <- df_orig %>%
  mutate(key = paste(year, county_name, crop, lat_lon, sep = "|"))

full_join_key <- full_join_df %>%
  mutate(key = paste(year, county_name, crop, lat_lon, sep = "|"))

# Find which keys from the original are not present in the full_join result
missing_keys <- setdiff(df_orig_key$key, full_join_key$key)
missing_rows <- df_orig_key %>% filter(key %in% missing_keys)

n_missing <- nrow(missing_rows)
cat("Number of missing rows in full_join_df relative to original data:", n_missing, "\n")
```

```
## Number of missing rows in full_join_df relative to original data: 44
```

These missing rows are explicit missing data because we can identify exactly which observations (the outliers) were removed based on a clear rule (irrigation demand $>$ 90th percentile per county-crop group).

Now we create a complete dataset where missing irrigation demand values are filled with the average demand for the corresponding county-crop combination (calculated from the non-outlier data).

```r
# Compute the average irrigation demand per county and crop using the filtered data
avg_demand <- df_filtered %>%
  group_by(county_name, crop) %>%
  summarise(avg_irrigation_demand = mean(irrigation_demand, na.rm = TRUE), .groups = "drop")

# For the missing rows, keep the original yield but replace irrigation_demand with the group average
missing_filled <- missing_rows %>%
  select(-key, -irrigation_demand) %>%             # remove temporary key and original demand
  left_join(avg_demand, by = c("county_name", "crop")) %>%
  rename(irrigation_demand = avg_irrigation_demand) %>%
  mutate(irrigation_demand = round(irrigation_demand, 2))  # round for readability
```

```r
# Combine the filtered data (full_join_df) with the filled missing rows
complete_dataset <- bind_rows(full_join_df, missing_filled) %>%
  arrange(year, county_name, crop, lat_lon)

# Verify that we now have all original rows
cat("Rows in complete dataset:", nrow(complete_dataset), "\n")
```

```
## Rows in complete dataset: 474
```

```r
cat("Rows in original dataset:", nrow(df_orig), "\n")
```

```
## Rows in original dataset: 474
```

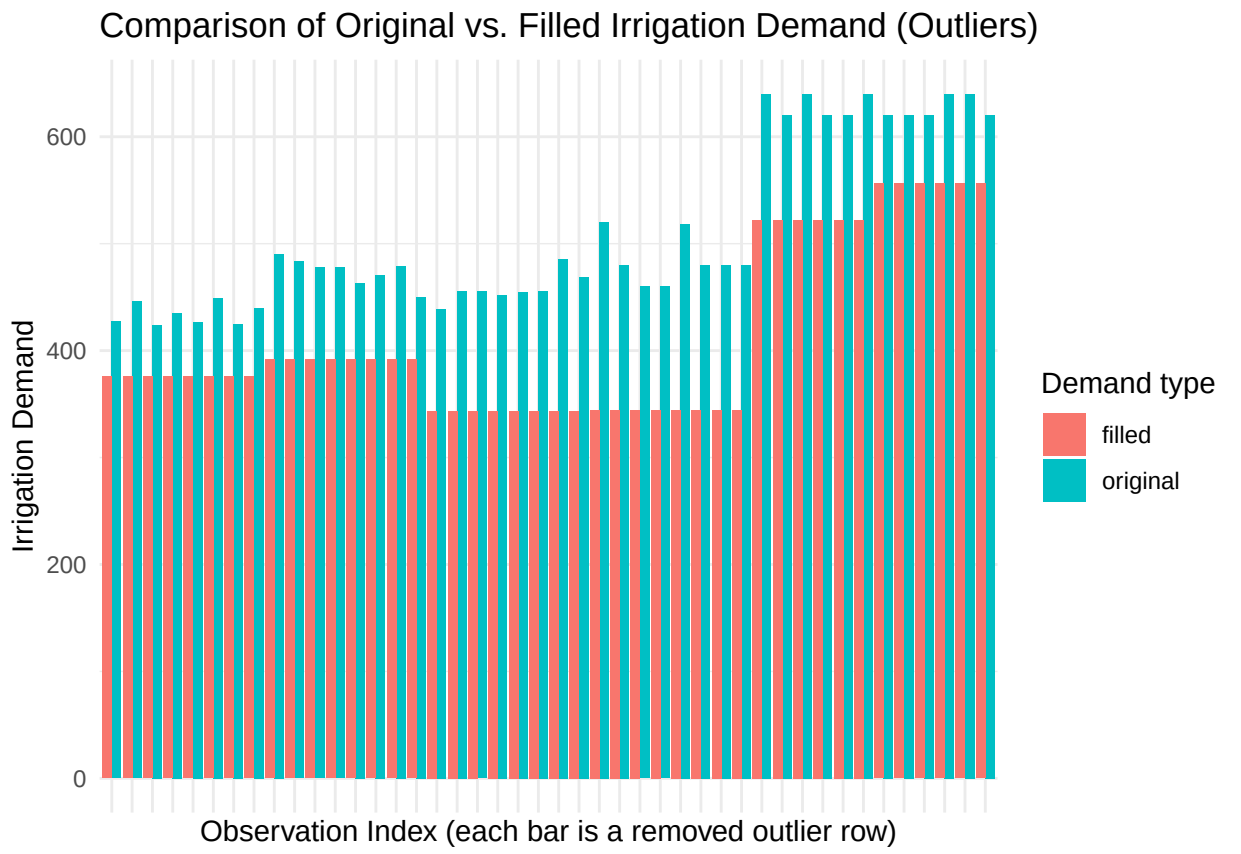
Step 6: Data frame of only the missing rows and visual comparison We create a data frame that contains only the missing rows, showing both the original irrigation demand (the outlier) and the filled-in average demand. Then we plot them to compare.

```r
# Create a comparison data frame with original and filled demand for missing rows
missing_comparison <- missing_rows %>%
  select(year, county_name, crop, lat_lon, yield,
         irrigation_demand_original = irrigation_demand) %>%
  left_join(avg_demand, by = c("county_name", "crop")) %>%
  rename(irrigation_demand_filled = avg_irrigation_demand) %>%
  mutate(irrigation_demand_filled = round(irrigation_demand_filled, 2))

# Add a unique row index so we can plot each observation separately
missing_comparison <- missing_comparison %>%
  mutate(row_id = row_number())

# Reshape the data from wide to long format: one column for demand type (original/filled) and one for v
comparison_long <- missing_comparison %>%
  select(row_id, county_name, crop, year,
         original = irrigation_demand_original,
         filled = irrigation_demand_filled) %>%
  pivot_longer(cols = c(original, filled),
               names_to = "type",
               values_to = "demand")

# Create a bar plot comparing original (outlier) and filled (average) demands
ggplot(comparison_long, aes(x = factor(row_id), y = demand, fill = type)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Comparison of Original vs. Filled Irrigation Demand (Outliers)",
       x = "Observation Index (each bar is a removed outlier row)",
       y = "Irrigation Demand",
       fill = "Demand type") +
  theme_minimal() +
  theme(axis.text.x = element_blank(),   # hide individual indices because there are many
        axis.ticks.x = element_blank())
```



Insights from the visual comparison: The original irrigation demands (red/orange bars) are consistently much higher than the filled values (blue bars).

The filled values are all moderate and lie close to the typical range of non-outlier data.

This confirms that the 90th percentile rule successfully identifies extreme demand values. Imputing with group averages makes the dataset complete but substantially reduces demand for those observations, which is important to remember in future analyses.

Conclusion We have completed all six tasks:

Added a year column.

Removed outliers above the 90th percentile per county-crop.

Created separate CSV files for irrigation demand and yield.

Performed full, left, and right joins (all same size due to perfect matching).

Identified missing rows as explicit missing data and filled them with county-crop averages.

Created a comparison data frame and a bar plot showing original vs. filled irrigation demands for the outliers.

The complete dataset is stored in complete_dataset and the missing-only comparison is in missing_comparison.